Funciones y Clases

Ejercicios entrega – R

Lina Quintero Fonseca

2026-03-02

Tabla de contenidos

1 Introducción **1**

2 Configuración inicial: la máquina matemática **2**

3 Ejercicio 1 – El cazador de coordenadas **2**
3.1 Descripción 2
3.2 Implementación de `calcular_bbox()` 2
3.3 Prueba de la función 3
3.4 Visualización de la Bounding Box en mapa interactivo 4

4 Ejercicio 2 – Modelando una ruta espacial **6**
4.1 Descripción 6
4.2 Clase `PuntoEspacial` 6
4.3 Clase `RutaGeografica` 7
4.4 Ejecución completa 10
4.5 Mapa interactivo de la ruta 12

5 Preguntas analíticas **13**
5.1 Sobre el Ejercicio 1: Regla DRY y pureza 13
5.2 Sobre el Ejercicio 2: Paradigmas de POO 14
5.3 Pregunta general: eficiencia y motores en Julia 14

6 Resumen **14**

1 Introducción

Este documento presenta la solución a los dos ejercicios del Capítulo 9 del curso. El objetivo es demostrar el uso de **funciones puras**, el sistema de clases **S3 de R**, y los principios de **abstracción** y **encapsulamiento** en un contexto geoespacial.

---

2 Configuración inicial: la máquina matemática

Antes de construir cualquier función o clase, traemos la herramienta de cálculo de distancias: la **fórmula de Haversine**. Esta función es la base matemática de todos los ejercicios.

```
library(leaflet)
library(ggplot2)
library(sf)
library(rnaturalearth)
library(rnaturalearthdata)

# --- FUNCIÓN BASE: HAVERSINE ---
# RECIBE: lat1, lon1, lat2, lon2 en grados decimales; radio en km (por defecto 6371)
# DEVUELVE: distancia en la unidad del radio suministrado
haversine <- function(lat1, lon1, lat2, lon2, radio = 6371.0) {
  rad <- function(g) g * pi / 180
  a <- sin(rad(lat2 - lat1) / 2)^2 +
       cos(rad(lat1)) * cos(rad(lat2)) * sin(rad(lon2 - lon1) / 2)^2
  return(radio * (2 * atan2(sqrt(a), sqrt(1 - a))))
}
```

---

3 Ejercicio 1 – El cazador de coordenadas

3.1 Descripción

En geomática necesitamos frecuentemente calcular la **Bounding Box** (caja delimitadora) de un conjunto de puntos para encuadrar un mapa o definir una región de análisis. Este ejercicio construye una función pura que recorre una lista de coordenadas y devuelve sus cuatro extremos geográficos.

3.2 Implementación de `calcular_bbox()`

```
# FUNCIÓN PURA: calcular_bbox
# -------------------------------------------------------
# RECIBE:
#   lista_coords → lista de vectores numéricos c(latitud, longitud)
# DEVUELVE:
#   lista con cuatro valores:
#     $lat_min  → latitud mínima del conjunto
#     $lat_max  → latitud máxima del conjunto
#     $lon_min  → longitud mínima del conjunto
#     $lon_max  → longitud máxima del conjunto
# -------------------------------------------------------
```

```r
# PRINCIPIO DE DISEÑO: función pura (no modifica variables
# externas; recibe datos y devuelve un resultado).
# -------------------------------------------------------
calcular_bbox <- function(lista_coords) {

  # Extraemos cada componente usando sapply para recorrer la lista
  latitudes  <- sapply(lista_coords, function(p) p[1])
  longitudes <- sapply(lista_coords, function(p) p[2])

  # Calculamos los cuatro extremos y los empaquetamos en una lista
  return(list(
    lat_min = min(latitudes),
    lat_max = max(latitudes),
    lon_min = min(longitudes),
    lon_max = max(longitudes)
  ))
}
```

3.3 Prueba de la función

```r
# Definimos un conjunto de coordenadas de ciudades colombianas
puntos_colombia <- list(
  c(4.6097,  -74.0817),   # Bogotá
  c(6.2442,  -75.5812),   # Medellín
  c(3.4516,  -76.5320),   # Cali
  c(10.3997, -75.4795),   # Cartagena
  c(7.8939,  -72.5078),   # Cúcuta
  c(1.2136,  -77.2811)    # Pasto
)

bbox <- calcular_bbox(puntos_colombia)

cat("                      \n")
```

```r
cat("         BOUNDING BOX CALCULADA         \n")
```

```
         BOUNDING BOX CALCULADA
```

```r
cat("                      \n")
```

```r
cat(sprintf("   Latitud  mínima : %8.4f°           \n", bbox$lat_min))
```

```
   Latitud  mínima :   1.2136°
```

```r
cat(sprintf("   Latitud  máxima : %8.4f°          \n", bbox$lat_max))
```

```
   Latitud  máxima :  10.3997°
```

```r
cat(sprintf("   Longitud mínima : %8.4f°          \n", bbox$lon_min))
```

```
   Longitud mínima : -77.2811°
```

```r
cat(sprintf("   Longitud máxima : %8.4f°          \n", bbox$lon_max))
```

```
   Longitud máxima : -72.5078°
```

```r
cat("                      \n")
```

3.4 Visualización de la Bounding Box en mapa interactivo

```r
# Extraemos latitudes y longitudes para los marcadores
lats    <- sapply(puntos_colombia, function(p) p[1])
lons    <- sapply(puntos_colombia, function(p) p[2])
nombres <- c("Bogota", "Medellin", "Cali", "Cartagena", "Cucuta", "Pasto")

if (knitr::is_html_output()) {
  # HTML: mapa interactivo leaflet
  leaflet() %>%
    addTiles() %>%
    addCircleMarkers(
      lng = lons, lat = lats,
      popup = nombres, label = nombres,
      radius = 6, color = "#2c7bb6", fillOpacity = 0.9
    ) %>%
    addRectangles(
      lng1 = bbox$lon_min, lat1 = bbox$lat_min,
      lng2 = bbox$lon_max, lat2 = bbox$lat_max,
      color = "#d7191c", weight = 2, fillOpacity = 0.05
    ) %>%
```

```r
    addLegend(
      position = "bottomright",
      colors = c("#2c7bb6", "#d7191c"),
      labels = c("Ciudades", "Bounding Box"),
      title = "Leyenda"
    )
} else {
  # PDF: mapa estatico con ggplot2
  mundo <- ne_countries(scale = "medium", returnclass = "sf")

  df_puntos <- data.frame(
    nombre = nombres,
    lat = lats,
    lon = lons
  )

  ggplot() +
    geom_sf(data = mundo, fill = "#f0f0f0", color = "#cccccc") +
    geom_rect(
      aes(xmin = bbox$lon_min, xmax = bbox$lon_max,
          ymin = bbox$lat_min, ymax = bbox$lat_max),
      fill = NA, color = "#d7191c", linewidth = 0.8, linetype = "dashed"
    ) +
    geom_point(data = df_puntos, aes(x = lon, y = lat),
               color = "#2c7bb6", size = 3) +
    geom_text(data = df_puntos, aes(x = lon, y = lat, label = nombre),
              vjust = -0.8, size = 2.8, color = "#1a1a1a") +
    coord_sf(
      xlim = c(bbox$lon_min - 2, bbox$lon_max + 2),
      ylim = c(bbox$lat_min - 2, bbox$lat_max + 2)
    ) +
    labs(
      title = "Bounding Box - Ciudades de Colombia",
      subtitle = "Rectangulo delimitador calculado con calcular_bbox()",
      x = "Longitud", y = "Latitud",
      caption = "Fuente: coordenadas ingresadas manualmente"
    ) +
    theme_minimal(base_size = 11) +
    theme(
      plot.title    = element_text(face = "bold"),
      panel.grid    = element_line(color = "#eeeeee")
    )
}
```

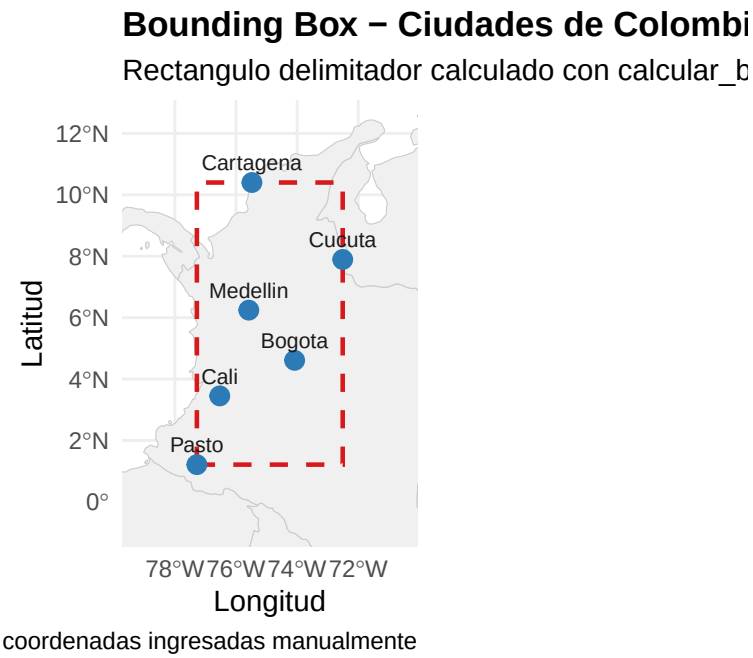

Figura 1: Figura E1.1 - Bounding Box de las ciudades colombianas calculada con calcular_bbox()

---

4 Ejercicio 2 – Modelando una ruta espacial

4.1 Descripción

Este ejercicio construye dos clases S3 que interactúan entre sí: `PuntoEspacial` y la nueva clase `RutaGeografica`, que encapsula una secuencia de puntos y tiene la capacidad de calcular la distancia total recorrida.

4.2 Clase `PuntoEspacial`

```
# ============================================================
# CLASE S3: PuntoEspacial
# Molde: lista etiquetada con latitud, longitud y nombre
# ============================================================

# [1] CONSTRUCTOR INFORMAL
nuevo_punto_espacial <- function(latitud, longitud, nombre = "Punto sin nombre") {
  objeto <- list(latitud = latitud, longitud = longitud, nombre = nombre)
  class(objeto) <- "PuntoEspacial"
  return(objeto)
}
```

```r
# [2] REPRESENTACIÓN TEXTUAL
print.PuntoEspacial <- function(obj, ...) {
  cat(sprintf("[%s]  Lat: %.4f  |  Lon: %.4f\n",
              obj$nombre, obj$latitud, obj$longitud))
}

# [3] MÉTODO ANALÍTICO: distancia_hacia
# Función genérica
distancia_hacia <- function(obj_origen, obj_destino, radio = 6371.0) {
  UseMethod("distancia_hacia")
}

# Implementación específica para PuntoEspacial
# RECIBE:  dos objetos PuntoEspacial + radio opcional
# DEVUELVE: distancia en km (o en la unidad del radio)
distancia_hacia.PuntoEspacial <- function(obj_origen, obj_destino,
                                          radio = 6371.0) {
  return(haversine(
    obj_origen$latitud,  obj_origen$longitud,
    obj_destino$latitud, obj_destino$longitud,
    radio = radio
  ))
}
```

4.3 Clase `RutaGeografica`

```r
# ============================================================
# CLASE S3: RutaGeografica
# Molde: lista etiquetada con un nombre y una colección
#         de objetos PuntoEspacial ordenados en secuencia.
# ============================================================

# [1] CONSTRUCTOR
# RECIBE:  nombre (string) de la ruta
# DEVUELVE: objeto RutaGeografica con lista de puntos vacía
nueva_ruta_geografica <- function(nombre) {
  objeto <- list(
    nombre = nombre,
    puntos = list()        # Inicia vacía; se llena con agregar_punto()
  )
  class(objeto) <- "RutaGeografica"
  return(objeto)
}

# [2] REPRESENTACIÓN TEXTUAL
```

```r
print.RutaGeografica <- function(obj, ...) {
  cat(sprintf("Ruta: '%s'  |  %d punto(s) registrado(s)\n",
              obj$nombre, length(obj$puntos)))
}

# [3] MÉTODO: agregar_punto
# Función genérica
agregar_punto <- function(ruta, punto) {
  UseMethod("agregar_punto")
}

# RECIBE:  objeto RutaGeografica + un PuntoEspacial
# DEVUELVE: la ruta actualizada con el nuevo punto añadido al final
agregar_punto.RutaGeografica <- function(ruta, punto) {
  ruta$puntos <- c(ruta$puntos, list(punto))
  return(ruta)
}

# [4] MÉTODO: medir_distancia_total
# Función genérica
medir_distancia_total <- function(ruta) {
  UseMethod("medir_distancia_total")
}

# RECIBE:  objeto RutaGeografica con >= 2 puntos
# DEVUELVE: distancia total acumulada en km
# LÓGICA:  recorre los tramos i → i+1 usando distancia_hacia()
#          que ya pertenece al propio PuntoEspacial (encapsulamiento)
medir_distancia_total.RutaGeografica <- function(ruta) {
  n <- length(ruta$puntos)

  if (n < 2) {
    message("La ruta necesita al menos 2 puntos para calcular distancia.")
    return(invisible(0))
  }

  total <- 0
  for (i in 1:(n - 1)) {
    d <- distancia_hacia(ruta$puntos[[i]], ruta$puntos[[i + 1]])
    total <- total + d
  }
  return(total)
}

# [5] MÉTODO GRÁFICO: graficar_ruta
# Función genérica
graficar_ruta <- function(ruta) {
```

```r
  UseMethod("graficar_ruta")
}

# RECIBE:  objeto RutaGeografica con >= 2 puntos
# DEVUELVE: mapa leaflet interactivo con marcadores,
#           polilínea de la ruta y popup con distancia total
graficar_ruta.RutaGeografica <- function(ruta) {
  n <- length(ruta$puntos)

  lats    <- sapply(ruta$puntos, function(p) p$latitud)
  lons    <- sapply(ruta$puntos, function(p) p$longitud)
  nombres <- sapply(ruta$puntos, function(p) p$nombre)

  # Distancia total
  dist_total <- medir_distancia_total(ruta)

  # Punto medio geográfico para anclar la etiqueta de distancia
  mid_lat <- mean(lats)
  mid_lon <- mean(lons)

  # Construimos el mapa
  mapa <- leaflet() %>%
    addTiles() %>%
    # Línea de la ruta
    addPolylines(
      lng    = lons,
      lat    = lats,
      color  = "#d7191c",
      weight = 3,
      opacity = 0.85,
      label  = sprintf("Ruta: %s", ruta$nombre)
    ) %>%
    # Marcadores de cada punto
    addCircleMarkers(
      lng         = lons,
      lat         = lats,
      popup       = sprintf("<b>%s</b><br>Lat: %.4f<br>Lon: %.4f",
                            nombres, lats, lons),
      label       = nombres,
      radius      = 7,
      color       = "#1a1a1a",
      fillColor   = "#2c7bb6",
      fillOpacity = 0.9,
      weight      = 2
    ) %>%
    # Marcador de inicio (verde) y fin (rojo)
    addCircleMarkers(
```

```r
      lng = lons[1], lat = lats[1],
      radius = 9, color = "#1a6400", fillColor = "#4dac26",
      fillOpacity = 1, weight = 2,
      popup = paste0("<b>Inicio:</b> ", nombres[1])
    ) %>%
    addCircleMarkers(
      lng = lons[n], lat = lats[n],
      radius = 9, color = "#7b0000", fillColor = "#d7191c",
      fillOpacity = 1, weight = 2,
      popup = paste0("<b>Fin:</b> ", nombres[n])
    ) %>%
    # Etiqueta con la distancia total en el punto medio
    addPopups(
      lng   = mid_lon,
      lat   = mid_lat,
      popup = sprintf(
        "<b>Ruta:</b> %s<br><b>Distancia total:</b> %.2f km<br><b>Tramos:</b> %d",
        ruta$nombre, dist_total, n - 1
      )
    ) %>%
    addLegend(
      position = "bottomright",
      colors   = c("#4dac26", "#2c7bb6", "#d7191c"),
      labels   = c("Punto de inicio", "Puntos intermedios", "Punto final / Ruta"),
      title    = ruta$nombre
    )

  return(mapa)
}
```

4.4 Ejecución completa

```r
# [1] Instanciamos los PuntoEspacial
p1 <- nuevo_punto_espacial(4.6097,   -74.0817, "Bogotá")
p2 <- nuevo_punto_espacial(6.2442,   -75.5812, "Medellín")
p3 <- nuevo_punto_espacial(3.4516,   -76.5320, "Cali")
p4 <- nuevo_punto_espacial(10.3997,  -75.4795, "Cartagena")
p5 <- nuevo_punto_espacial(7.8939,   -72.5078, "Cúcuta")

# [2] Creamos la ruta vacía
ruta_colombia <- nueva_ruta_geografica("Ruta Colombia")

# [3] Agregamos los puntos en orden
ruta_colombia <- agregar_punto(ruta_colombia, p1)
ruta_colombia <- agregar_punto(ruta_colombia, p2)
```

```r
ruta_colombia <- agregar_punto(ruta_colombia, p3)
ruta_colombia <- agregar_punto(ruta_colombia, p4)
ruta_colombia <- agregar_punto(ruta_colombia, p5)

# [4] Representación textual de la ruta
print(ruta_colombia)
```

```
Ruta: 'Ruta Colombia'  |  5 punto(s) registrado(s)
```

```r
# [5] Desglose por tramos
cat("\nDesglose por tramos:\n")
```

```
Desglose por tramos:
```

```r
cat(paste(rep("-", 50), collapse = ""), "\n")
```

```
--------------------------------------------------
```

```r
n <- length(ruta_colombia$puntos)
for (i in 1:(n - 1)) {
  d <- distancia_hacia(ruta_colombia$puntos[[i]], ruta_colombia$puntos[[i + 1]])
  cat(sprintf("  Tramo %d | %-12s -> %-12s | %7.2f km\n",
              i,
              ruta_colombia$puntos[[i]]$nombre,
              ruta_colombia$puntos[[i + 1]]$nombre,
              d))
}
```

```
  Tramo 1 | Bogotá      -> Medellín    |  246.14 km
  Tramo 2 | Medellín    -> Cali        |  327.90 km
  Tramo 3 | Cali         -> Cartagena    |  781.27 km
  Tramo 4 | Cartagena    -> Cúcuta      |  429.01 km
```

```r
cat(paste(rep("-", 50), collapse = ""), "\n")
```

```
--------------------------------------------------
```

```r
# [6] Distancia total
total <- medir_distancia_total(ruta_colombia)
cat(sprintf("\n  TOTAL  | %-28s | %7.2f km\n", ruta_colombia$nombre, total))
```

```

  TOTAL  | Ruta Colombia                | 1784.31 km
```

4.5 Mapa interactivo de la ruta

```r
if (knitr::is_html_output()) {
  # HTML: mapa interactivo leaflet
  graficar_ruta(ruta_colombia)
} else {
  # PDF: mapa estatico con ggplot2
  mundo <- ne_countries(scale = "medium", returnclass = "sf")

  lats_r    <- sapply(ruta_colombia$puntos, function(p) p$latitud)
  lons_r    <- sapply(ruta_colombia$puntos, function(p) p$longitud)
  nombres_r <- sapply(ruta_colombia$puntos, function(p) p$nombre)

  df_ruta <- data.frame(lon = lons_r, lat = lats_r, nombre = nombres_r)
  dist_total <- medir_distancia_total(ruta_colombia)

  ggplot() +
    geom_sf(data = mundo, fill = "#f0f0f0", color = "#cccccc") +
    geom_path(data = df_ruta, aes(x = lon, y = lat),
              color = "#d7191c", linewidth = 1.2) +
    geom_point(data = df_ruta, aes(x = lon, y = lat),
               color = "#2c7bb6", size = 3.5) +
    geom_point(data = df_ruta[1, ],
               aes(x = lon, y = lat),
               color = "#4dac26", size = 5) +
    geom_point(data = df_ruta[nrow(df_ruta), ],
               aes(x = lon, y = lat),
               color = "#d7191c", size = 5) +
    geom_text(data = df_ruta, aes(x = lon, y = lat, label = nombre),
              vjust = -0.9, size = 2.8, color = "#1a1a1a") +
    coord_sf(
      xlim = c(min(lons_r) - 2, max(lons_r) + 2),
      ylim = c(min(lats_r) - 2, max(lats_r) + 2)
    ) +
    labs(
      title = paste("Ruta:", ruta_colombia$nombre),
      subtitle = sprintf("Distancia total: %.2f km | %d tramos",
                         dist_total, length(ruta_colombia$puntos) - 1),
      x = "Longitud", y = "Latitud",
      caption = "Inicio (verde) -> Fin (rojo)"
    ) +
    theme_minimal(base_size = 11) +
    theme(
      plot.title    = element_text(face = "bold"),
      plot.subtitle = element_text(color = "#555555"),
      panel.grid    = element_line(color = "#eeeeee")
    )
```

```
}
```

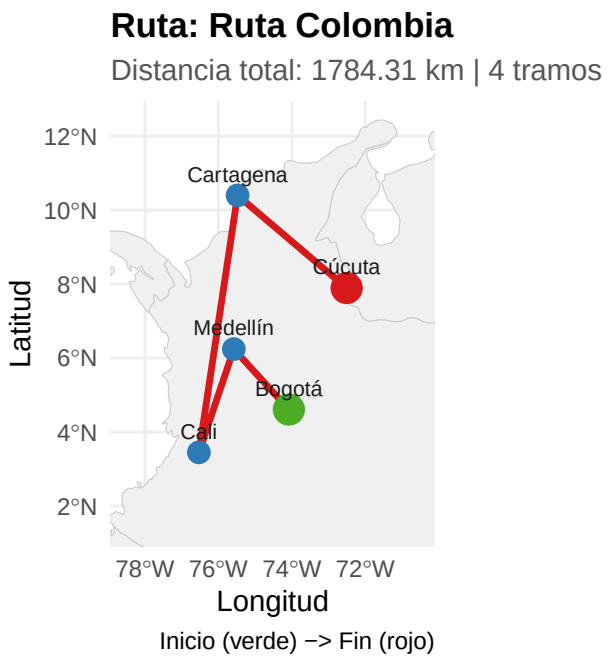

Figura 2: Figura E2.1 - Ruta geografica generada con la clase RutaGeografica

---

5 Preguntas analíticas

5.1 Sobre el Ejercicio 1: Regla DRY y pureza

La regla **DRY** (*Don't Repeat Yourself*) establece que cada pieza de lógica debe existir en un único lugar del código. Si se copia y pega la fórmula de Haversine en múltiples partes del proyecto, cualquier corrección futura (por ejemplo, ajustar el radio de la Tierra o cambiar la función trigonométrica) debe hacerse en todos los lugares donde fue pegada. Este riesgo se multiplica con el tamaño del proyecto: basta con olvidar actualizar una sola copia para introducir resultados inconsistentes que son difíciles de rastrear. Al encapsularla en `haversine()`, existe un único punto de verdad que todos los demás módulos consultan.

La función `calcular_bbox()` también ilustra el concepto de **pureza**: recibe datos, opera internamente con ellos y devuelve un resultado sin tocar variables del entorno global. Esto la hace predecible, fácil de testear de forma aislada y segura para usarse en paralelo o dentro de ciclos masivos.

5.2 Sobre el Ejercicio 2: Paradigmas de POO

En **Python** (POO clásica), los métodos como `medir_distancia_total()` **viven físicamente dentro del objeto**, grabados en la clase como atributos del molde de hierro. Cuando se instancia un objeto, este carga consigo su propia copia de todas las herramientas.

En el **Sistema S3 de R**, la situación es radicalmente distinta: la clase `RutaGeografica` no es más que una lista común con un *post-it* pegado (`class(obj) <- "RutaGeografica"`). Los métodos **flotan libremente** en el entorno como funciones independientes. Cuando se llama a `medir_distancia_total(ruta)`, R actúa como un inspector de correos que lee la etiqueta del objeto, busca en su directorio la función `medir_distancia_total.RutaGeografica` y le entrega el paquete. El objeto en sí no sabe hacer nada; es el sistema de despacho quien organiza el trabajo.

5.3 Pregunta general: eficiencia y motores en Julia

Si `RutaGeografica` almacenara 5 millones de puntos GPS, R repetiría el proceso de leer el *post-it* y buscar en el directorio de métodos en **cada una de las 4,999,999 iteraciones** del ciclo, lo que lo hace lento para tareas científicas masivas.

Julia resolvería esto de forma diferente gracias a la **Compilación JIT** (*Just-In-Time*). La primera vez que se invoca `distancia_hacia(p1::PuntoEspacial, p2::PuntoEspacial)`, el compilador escanea los tipos exactos de ambos argumentos, genera **código máquina nativo** optimizado para esa combinación específica y construye un "tubo de ensamblaje" directo en memoria. Las siguientes 4,999,998 llamadas simplemente fluyen por ese tubo sin ningún costo de despacho, alcanzando velocidades comparables a C o Fortran.

---

6 Resumen

Concepto	Implementación en este documento
Función pura	`calcular_bbox()` – no modifica el entorno
Regla DRY	`haversine()` usada por todos los métodos
Clase S3 – Constructor	`nuevo_punto_espacial()`, `nueva_ruta_geografica()`
Clase S3 – Representación	`print.PuntoEspacial`, `print.RutaGeografica`
Clase S3 – Despacho	`distancia_hacia()`, `agregar_punto()`, `medir_distancia_total()`
Encapsulamiento	`RutaGeografica` delega el cálculo a `PuntoEspacial`
Visualización	`graficar_ruta()` con `leaflet`
Bounding Box	`calcular_bbox()` con rectángulo en mapa